

Version Control and Collaboration in MLOps

Aishwariya Chakraborty

Version Control in Software Development

- Tracking and managing changes made to project files or folders over time
- Code versioning
- Key concepts:
 - Version, Repository, Commit, History
- Advantages:
 - Traceability, Reproducibility, Rollback, Debugging, Collaboration

- ✓ Code → deterministic output
- ✓ Git handles it well: text diffs, cheap branching, distributed
 - ✓ Collaboration solved: PRs, code review, CI/CD

The ML Reproducibility Problem

In traditional software, the code is the product. In ML, the code is just one of five moving parts.

ML Experiment = Code + Data + Configuration + Environment + Randomness

- Data: Same code + different data = different model
- Configuration: Hyperparameters matter as much as architecture
- Environment: GPU, CUDA version, data version, library version
- Randomness: Weight init, shuffling, dropout, non-deterministic GPU ops

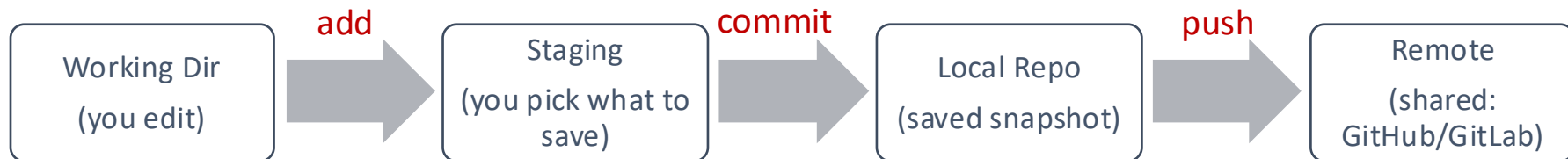
What needs versioning in ML?

Artifact	What it is	Why it's hard	Versioning Tool
Code	Training scripts, configs, pipeline definitions	-	Git
Data	Datasets, features, labels	Large, changes silently, too big for Git	DVC / Git LFS
Model	Trained weights, checkpoints	Large binaries, no meaningful diff	DVC / Git LFS / Model registry

Reproducibility isn't versioning these separately — it's capturing the exact combination (this code + this data + this model) as one recoverable point in time.

Git: Quick Overview

- Distributed version control system
 - Tracks and manages changes in code over time.
 - Allows multiple developers to work on the same project, view previous versions, and restore changes when needed.



Git: Quick Overview + Demo

- Distributed version control system
 - Tracks and manages changes in code over time.
 - Allows multiple developers to work on the same project, view previous versions, and restore changes when needed.

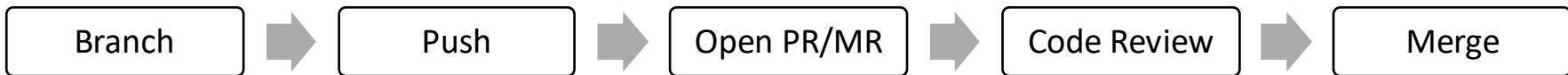
Setup & snapshot	Share & sync
<code>init</code> — start a repo	<code>clone</code> — copy a remote repo
<code>status</code> — what changed	<code>push</code> — send commits up
<code>add</code> — stage changes	<code>pull</code> — fetch + merge down
<code>commit</code> — save a snapshot	
<code>diff</code> — see exact changes	
<code>log</code> — history	

GitHub vs GitLab: Collaboration & .gitignore

- Platforms which add collaboration, review, and automation on top of the same Git.

	GitHub	GitLab
Propose + review changes	Pull Request	Merge Request
Track work & bugs	Issues	Issues
Build / test / deploy	Actions	Built-in CI/CD
Run on your own servers	Enterprise tier	First-class & free

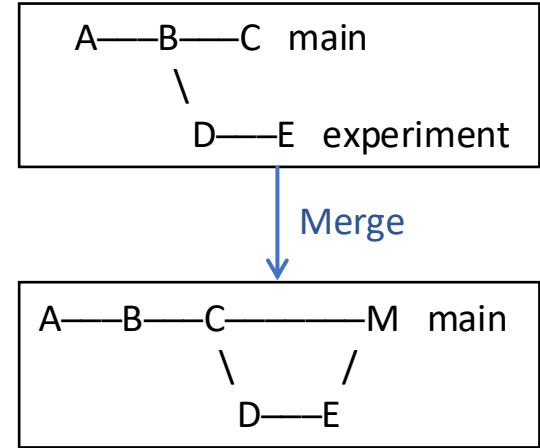
- Collaboration process:



- .gitignore – File that lists what Git should never track.
 - In ML, `__pycache__`, `.venv/`, `data/*.csv`, `*.h5` -- belong in DVC / Git LFS, not Git.

Branching & Merging

- Branch – moveable pointer to a commit – easy to create/delete
 - `git branch <name>` : create a branch
 - `git switch <name>` or `git checkout <name>` : move onto it
 - `git merge <name>` : merge a branch's work back into another
- Merge:
 - Clean merge – No overlapping edits – Git combines them



No-fast-forward	Fast-forward	Squash
Preserves experiment bubble	Linear, loses experiment boundary	One clean commit, noisy history discarded
Use for experiments	Use for hotfixes	Use for messy experiment branches

- Merge conflict – Two branches changed the same lines – Git asks you

Standard Branching Strategies

- A branching strategy is just a team agreement on when to branch and when to merge.
- Each branch = a feature (typically)
- Three common ones:

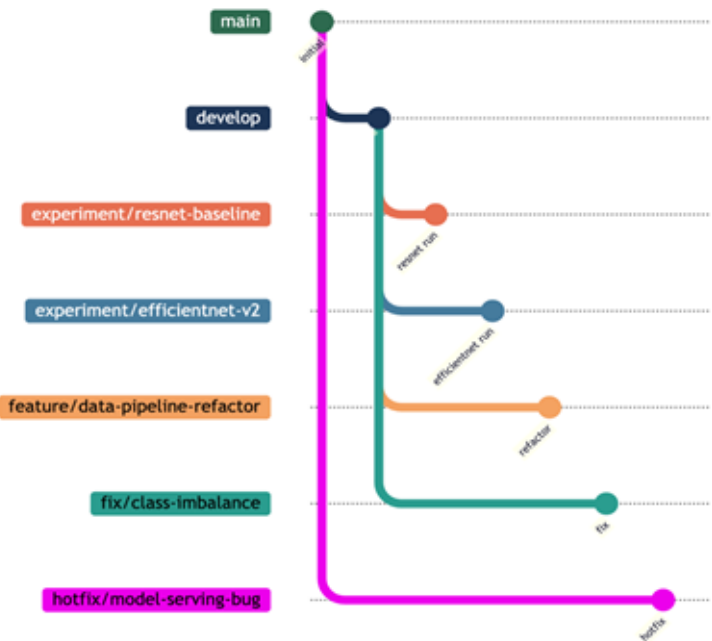
Strategy	How it works	Best for
Git Flow	Long-lived develop + release + feature branches	Scheduled releases, versioned software
GitHub Flow	One main + short-lived feature branches, merge via PR	Continuous delivery, web apps, most ML teams
Trunk Based	Everyone commits to main behind small, frequent merges	High-velocity teams with strong CI

- More long-lived branches = more isolation but more merge pain.
- Fewer branches = simpler but needs discipline + good CI.

Branching for ML

- ML branches isolate *experiments* and *pipeline* changes
 - Aim is to keep main reproducible and every metric change traceable to a single cause
- Experiment isolation:
 - One *hypothesis* per branch from same base commit
 - Compared against fixed baseline – *winner merged*
- Data pipeline changes and model changes kept on *separate* branches
 - data/<change> - preprocessing, features, dataset version, etc
 - model/<change> - architecture, loss, optimizer, hyperparameters, etc
- Released models marked with *annotated tags*
 - A tag is an immutable pointer to the exact commit
 - Combined with DVC, the tag pins code + data + config together – fully reconstructable model

Example ML branching strategy + Demo



Prefix	Purpose	Merge policy
experiment/	New approach / architecture / hyperparams	Only if it wins
feature/	Infrastructure, tooling, pipelines	PR + review
fix/	Bug in preprocessing / evaluation	Fast-track
data/	Data updates, schema changes	Needs data validation CI
release/	Release candidate	Only → main after validation

The Problem

- Git stores every version of every file in history
 - 200 MB model committed 10 times = ~2 GB repo, cloned by everyone, forever
- Binaries don't diff — Git can't delta-compress them, so each version is stored nearly in full.
- Result:
 - slow clones, bloated history, meaningless diffs — can't be deleted without rewriting history

```
git add model_weights.pt # 2.1 GB
git commit && git push    # Uploads 2.1 GB
# Retrain next epoch...
git commit && git push    # Another 2.1 GB
# 10 epochs: repo = 21 GB
# git clone = downloads ALL 21 GB
```

How Git LFS Solves It

- LFS - pointer in Git, blob on the server
 - Commit - Git stores a text pointer (a SHA-256 oid + size), not the file.
 - Push - the real blob goes to the LFS store
 - Clone/pull - blobs are fetched on demand.
- .gitattributes says which files this applies to — committed, so every collaborator gets the behavior automatically.

```
# Pointer stored by Git
version https://git-lfs.github.com/spec/v1
oid sha256:c036cbb7...
size 5242880
```

Git LFS Setup + Useful Commands + Demo

```
# Install (once per machine)
brew install git-lfs      # macOS
apt-get install git-lfs   # Ubuntu
git lfs install           # Initialize for user
```

```
# Track file types in repo
git lfs track "*.pt"      # PyTorch models
git lfs track "*.h5"      # Keras models
git lfs track "*.pkl"     # Pickled objects
git lfs track "*.parquet" # Data files
```

```
# Always commit .gitattributes
git add .gitattributes
git commit -m "Configure Git LFS"
```

```
# Use normally - LFS handles it transparently
git add model.pt
git commit -m "Add trained model"
git push
```

```
# What's tracked?
git lfs ls-files
```

```
# LFS status
git lfs status
```

```
# Pull only specific paths
git lfs pull --include="models/production/*"
```

```
# Clone without downloading LFS files
GIT_LFS_SKIP_SMUDGE=1 git clone <repo>
```

```
# Migrate existing repo to LFS
git lfs migrate import --include="*.pt" --everything
git push --force    # WARNING: rewrites history
```

```
# Clean local LFS cache
git lfs prune
```

LFS limits and the DVC decision point

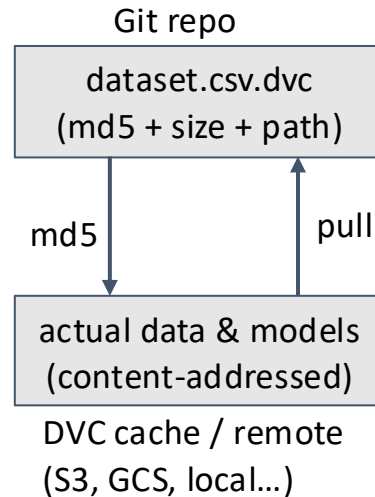
- Storage & bandwidth quotas
 - LFS storage is metered and billed (GitHub free tier: 1 GB storage + 1 GB/month bandwidth)
- Versions files, not datasets
 - Has no concept of directory of shards, train/validation/test split, or data lineage
- No experiment / stage awareness
 - Unable to express "this data produced this model via this step"
- Tied to your Git host
 - Does not support S3/GCS bucket
 - Data governance, egress cost, and access control not enabled

Scenario	Use
Small models, simple team	Git LFS
Large datasets (GB–TB)	DVC
Pipeline tracking needed	DVC
S3/GCS/Azure backend	DVC
Experiment comparison	DVC

Git LFS solves "large files in Git." It does not solve "version my data and pipeline."
That gap is exactly what DVC fills.

DVC — the missing layer

- "Git for data and models"
 - Git tracks a tiny metafile
 - DVC stores the real bytes in storage
 - Same commit/branch/checkout applied to multi-GB artifacts
- Concepts:
 - .dvc file - Small text metafile tracked by Git, holds the content hash + path of the real artifact
 - Cache - Local content-addressed store (.dvc/cache); working files are links into it
 - Remote - Your backend (S3 / GCS / Azure / SSH / local dir) where blobs are pushed & pulled



git add dataset.csv.dvc → version the pointer; dvc push → store the bytes
git checkout → moves the pointer back; dvc checkout → syncs the bytes to match

Getting started with DVC + Demo

```
pip install dvc
pip install "dvc[s3]"      # S3 support

git init && dvc init
git commit -m "Initialize DVC"

# Configure remote
dvc remote add -d myremote s3://my-bucket/dvc-store
# Local for testing:
dvc remote add -d myremote /tmp/dvc-cache

# Track data
dvc add data/raw/train.csv
# → creates data/raw/train.csv.dvc (goes in Git)
# → adds data/raw/train.csv to .gitignore

git add data/raw/train.csv.dvc data/raw/.gitignore
git commit -m "Track training data"

dvc push      # Upload to remote
dvc pull      # Download on fresh clone
```

DVC Pipelines: dvc.yaml & dvc repro

- DVC Pipelines version the process — the reproducible steps that turn raw data into a model
- A pipeline is a graph of stages with inputs (deps) and outputs (outs) defined in dvc.yaml:

```
stages:  
  prepare:  
    cmd: python src/prepare.py  
    deps: [src/prepare.py, data/raw.csv]  
    outs: [data/dataset.csv]  
  train:  
    cmd: python src/train.py  
    deps: [src/train.py, data/dataset.csv]  
    outs: [models/model.h5]
```

- Based on this yaml, DVC builds a dependency graph
- dvc repro re-runs only the stages whose inputs changed:
 - In this example, if you change the data, and prepare + train re-run; if you change nothing, it's a no-op

Beyond the Basics: dvc.lock File & dvc exp

- How does dvc repro know if anything changed – dvc.lock file
 - Auto-generated by dvc repro.
 - Records the exact resolved hashes of every stage's deps, outs, and params at the last successful run.
 - Dvc repro compares the current hashes against dvc.lock.
 - Match → skip the stage. Differ → re-run it.
- Dvc.yaml + dvc.lock – committed to Git – together they pin the pipeline
- dvc exp — experiments without branch clutter
 - Each run is a lightweight, hidden commit
 - dvc exp show : gives a metrics-vs-params comparison table
 - Reproducible by construction: each experiment records its params and links to the dvc.lock state that produced it.

LFS vs DVC

	Git LFS	DVC
Primary job	Store large binaries in a Git repo	Version data, models and pipelines
Storage backend	Git host's LFS store (coupled)	Your S3 / GCS / Azure / SSH / local
Data lineage	File history only	Links data → stage → model
Pipelines / reproducibility	No	Yes, dvc.yaml + dvc repro
Experiment management	No	Yes, dvc exp
Setup cost	Trivial (git lfs track)	Slightly more (init, remote, .dvc files)
Best for	A few model checkpoints, artifacts in an existing repo	Real datasets, reproducible ML workflows

LFS for simple/small teams. DVC for reproducible pipelines at scale.

Best-Practices

- Version code + data + config together — a commit should fully describe an experiment
- .gitignore large files; track them with DVC/LFS, never raw Git
- One variable per branch/experiment — keep metric changes attributable
- Tag released models; log metrics to a tracker, not commit messages
- Always pair git push with dvc push
- Commit both dvc.yaml and dvc.lock so pipelines are reproducible

Because in machine learning, code without data is only half a commit